

Tickets

Moritz

2026-05-05

Contents

1 Bug Reporting	1
1.1 Adaptive maintenance	2
1.2 Corrective maintenance	3
1.3 Perfective maintenance	4
1.4 Preventive maintenance	5

1 Bug Reporting

Idealerweise besteht ein Bug Report aus:

1. Ticket-Nummer (id)
2. Titel
3. Priorität (mindestens 3 Stufen)
4. Autor
5. Version der Software
6. Umgebung (Betriebssystem, Hardware)
7. Beschreibung des Fehlers (was ist das Problem?)
8. Soll- und ist-Resultat (was verlangt die Spezifikation, wie ist die Realität)
9. Reproduktion des Fehlers (Testfall, einzelne Schritte)
10. ggf. Screenshots

1.1 Adaptive maintenance

Ticket-Nummer	1
Titel	JExam funktioniert unter iOS nicht
Priorität	Low
Autor	Moritz
Version	1.0.0
Umgebung	iOS 17.4 (Safari), Java 21 auf Server

Beschreibung des Fehlers

JExam ist laut Spezifikation eine JavaFX-Desktop-Anwendung für Windows, macOS und Linux. Auf iOS kann die Anwendung nicht ausgeführt werden, wodurch mobile Nutzung auf iPad/iPhone nicht möglich ist.

Soll- und ist-Resultat

Soll: Produktentscheidung für Adaptive Maintenance: Bereitstellung einer iOS-kompatiblen Nutzung (z.B. Web-Frontend oder Remote-Client), damit Lehrende auch mobil arbeiten können.

Ist: Der aktuelle Stand deckt nur Desktop-Plattformen ab; iOS wird nicht unterstützt.

Reproduktion des Fehlers

1. JExam-Release auf iPhone/iPad öffnen oder starten.
2. Keine native Ausführung möglich, da keine unterstützte Desktop-Laufzeit vorhanden ist.
3. Kernfunktionen (XML-Verwaltung, PDF-Generierung) sind auf iOS nicht nutzbar.

1.2 Corrective maintenance

Ticket-Nummer	2
Titel	Kapitel Reihenfolge ist nicht Persistent
Priorität	Medium
Autor	Moritz
Version	1.0.0
Umgebung	Windows 11, Java 21

Beschreibung des Fehlers

Die in der Klausurkonfiguration geänderte Reihenfolge der Kapitel (UC-14) wird nach Speichern und erneutem Laden der XML-Datei nicht korrekt wiederhergestellt. Nach dem Reload erscheint wieder die alte oder eine inkonsistente Reihenfolge.

Soll- und ist-Resultat

Soll: Die durch den Nutzer geänderte Kapitelreihenfolge muss über Speichern/Laden (UC-03, UC-02) persistent bleiben und identisch für die PDF-Generierung verwendet werden.

Ist: Reihenfolge geht nach Neustart oder Reload verloren.

Reproduktion des Fehlers

1. XML-Datei laden.
2. In die Klausurkonfiguration wechseln.
3. Kapitel mit UP/DOWN umordnen (UC-14).
4. Datei speichern (UC-03).
5. Datei schliessen und neu laden (UC-02).
6. Reihenfolge prüfen: entspricht nicht der gespeicherten Reihenfolge.

1.3 Perfective maintenance

Ticket-Nummer	3
Titel	Generierung von PDFs mit vielen Aufgaben dauert lange
Priorität	Medium
Autor	Moritz
Version	1.0.0
Umgebung	Ubuntu 24.04 Server, Java 21

Beschreibung des Fehlers

Bei grossen Aufgabenpools (mehrere hundert bis tausend Tasks/Variants) dauert die PDF-Generierung deutlich länger als erwartet. Die UI wirkt währenddessen blockiert und reagiert verzögert.

Soll- und ist-Resultat

Soll: Laut nicht-funktionaler Anforderung soll die PDF-Generierung in weniger als 5 Sekunden erfolgen.

Ist: Bei großen Datenmengen (1.000 Tasks) liegt die Laufzeit bei ca. 10 bis 20 Sekunden.

Reproduktion des Fehlers

1. Großes XML-Aufgabenpool (1.000 Tasks) mit vielen Chapters/Tasks laden.
2. "Klausur generieren (PDF)" starten (UC-15).
3. Zeit bis zur fertigen PDF messen.
4. Laufzeit liegt regelmässig über 5 Sekunden.

1.4 Preventive maintenance

Ticket-Nummer	4
Titel	Regressionen in der Bewertung durch fehlende automatisierte Tests vermeiden
Priorität	Medium
Autor	Moritz
Version	1.0.0
Umgebung	Java 21, JUnit 5

Beschreibung des Fehlers

Kritische Regeln (z.B. Validierung von Task-Feldern, Difficulty-Verteilung, Export-Pfade) sind nicht ausreichend durch automatisierte Tests abgesichert. Dadurch können Refactorings unerkannt Regressionen verursachen.

Soll- und ist-Resultat

Soll: Zentrale Use Cases und Validierungsregeln sind durch Unit- und Integrations-Tests abgesichert; fehlerhafte Änderungen werden vor Release erkannt.

Ist: Mehrere Kernpfade sind nur manuell getestet, Regressionen fallen spät auf.

Reproduktion des Fehlers

1. Kleine Änderung an Validierungslogik oder PDF-Filterung vornehmen.
2. Build/Testlauf starten.
3. Trotz fachlichem Fehler kein Test-Fail.
4. Erst manueller Test zeigt Fehlverhalten.